



현실적인 몽상가 송병국 입니다

송병국 1999.03.23

아이디어를 말로만 끝내지 않습니다.
최소한의 기능을 단위로 쪼개 우선순위를 세우고,
일정 안에 구현하기 위해 최선을 다 합니다.
문제가 생기면 재현부터 원인 분리까지
신중하게 접근해 재발을 줄입니다.
혼자 빠르게보다 함께 멀리 가는 쪽을 택해
배운 내용을 정리해 공유하며 성장해왔습니다.

프로필

전화번호 010.4658.5892
이메일 bksong121212@gmail.com
주소 김포 한강8로 173-88 (마산동)

교육 이력 및 경력 사항

최종학력 부천대학교 정보통신공학과 졸업
25.07 ~ 25.12 코리아 IT 아카데미
공공데이터 융합 풀스택 개발자 양성과정 O
20.04 ~ 22.04 부천대학교 학술정보원
교내 네트워크 및 PC 관리
19.12 ~ 20.03 부천대학교 정보통신과
실습실 관리 보조
2018 부천대학교 평생교육원
전화 응대 및 업무 보조

자격증

2022.12.13 네트워크관리사 2급

경험 및 프로젝트

25.11 ~ 25.12 EV119
응급실 정보 서비스
25.07 ~ 25.11 WebNest
10대 청소년들을 위한 프로그래밍 웹 서비스
19.07 ~ 19.09 캡스톤 디자인 : 부천대학교
아두이노를 활용한 드론 제작
19.07 ~ 19.08 현장 실습 : 아사달
자료를 조사하고 정리하여 웹 서버에 등록

사이트

깃허브 <https://github.com/ByoungGuk1>
 노션 <https://buly.kr/2JpFeBh>
 velog <https://velog.io/@bksong9903/posts>

활용 가능 기술

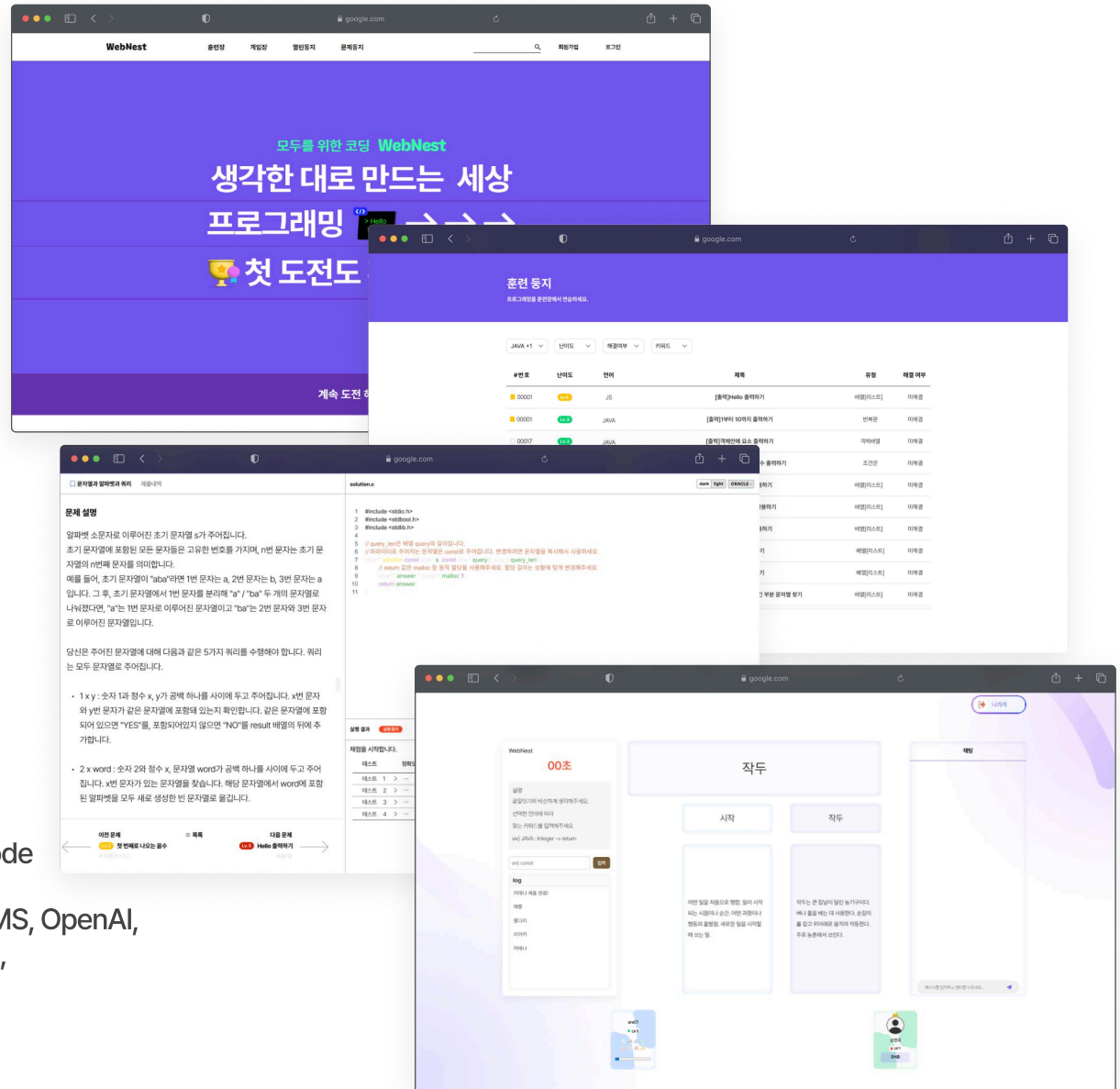
C, JAVA, HTML, CSS, JS, TS, React, Redux
Node, JSP, Spring, Spring Boot, STOMP
Spring Data JPA, Spring Security, JWT,
MyBatis, Swagger, Oracle, Redis, Docker, AWS,
Git, GitHub, Figma



- 개발 기간 2025. 07 ~ 2025. 11 (약 3개월)
- 플랫폼 Web
- 개발 인원 5명 (팀장)
- 담당 역할 로그인, 회원가입 페이지
아이디 & 비밀번호 찾기
멀티 게임방 사용자 프로필
멀티게임 - LLM을 활용한 끝말잇기

Development Environment

- Language Java (JDK 17), HTML, CSS, JavaScript
- Server, Cloud Apache Tomcat 9.0
- Framework Spring Boot 3.2.x, React 18.x
- DB Oracle 21C, Redis 8.x
- IDE IntelliJ IDEA 2025.2.3, Visual Studio Code
- API, Library STOMP(WebSocket), Swagger, coolSMS, OpenAI, Monaco Editor, Swiper API, Stream API, OAuth2, Spring Security, JWT
- DevOps, Tools Git, GitHub, Docker





외부 API



FrontEnd (React)



port : 3000

React



Redux

BackEnd (SpringBoot)



port : 10000

SpringBoot



Security



JWT



My-batis



Stomp



Apache



JAVA



Spring



Swagger UI

DataBase



Oracle



Redis

port : 1521

port : 6379

Request

Response

REST API

Client

Manage



GitHub



Jenkins

Plan



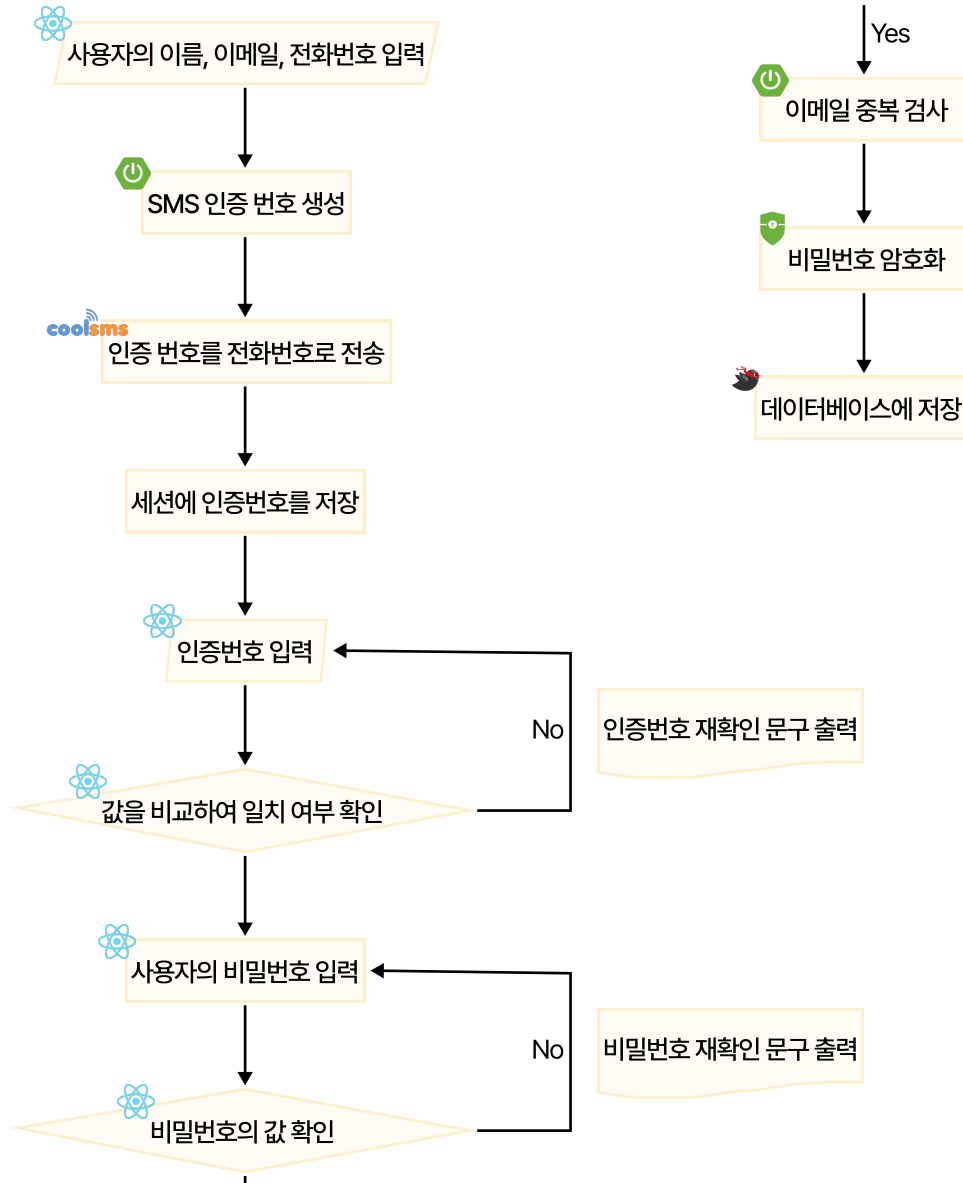
ERD



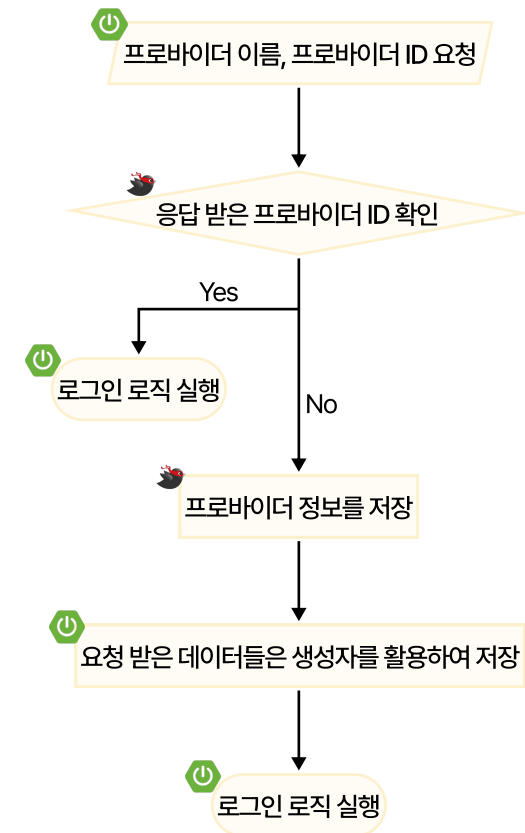
Google Cloud



일반 회원가입

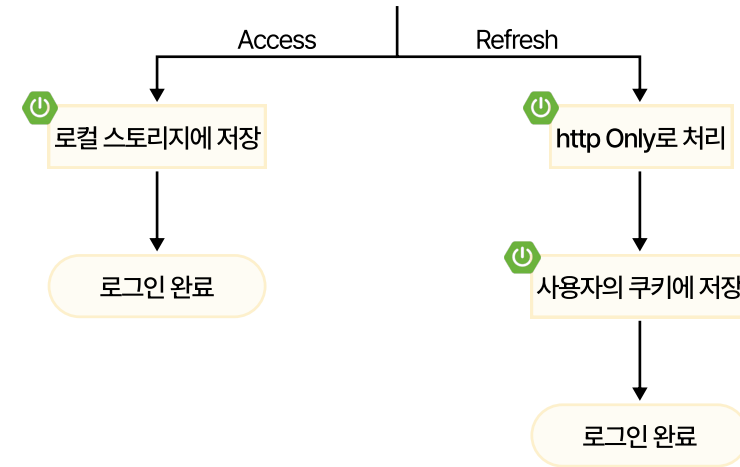
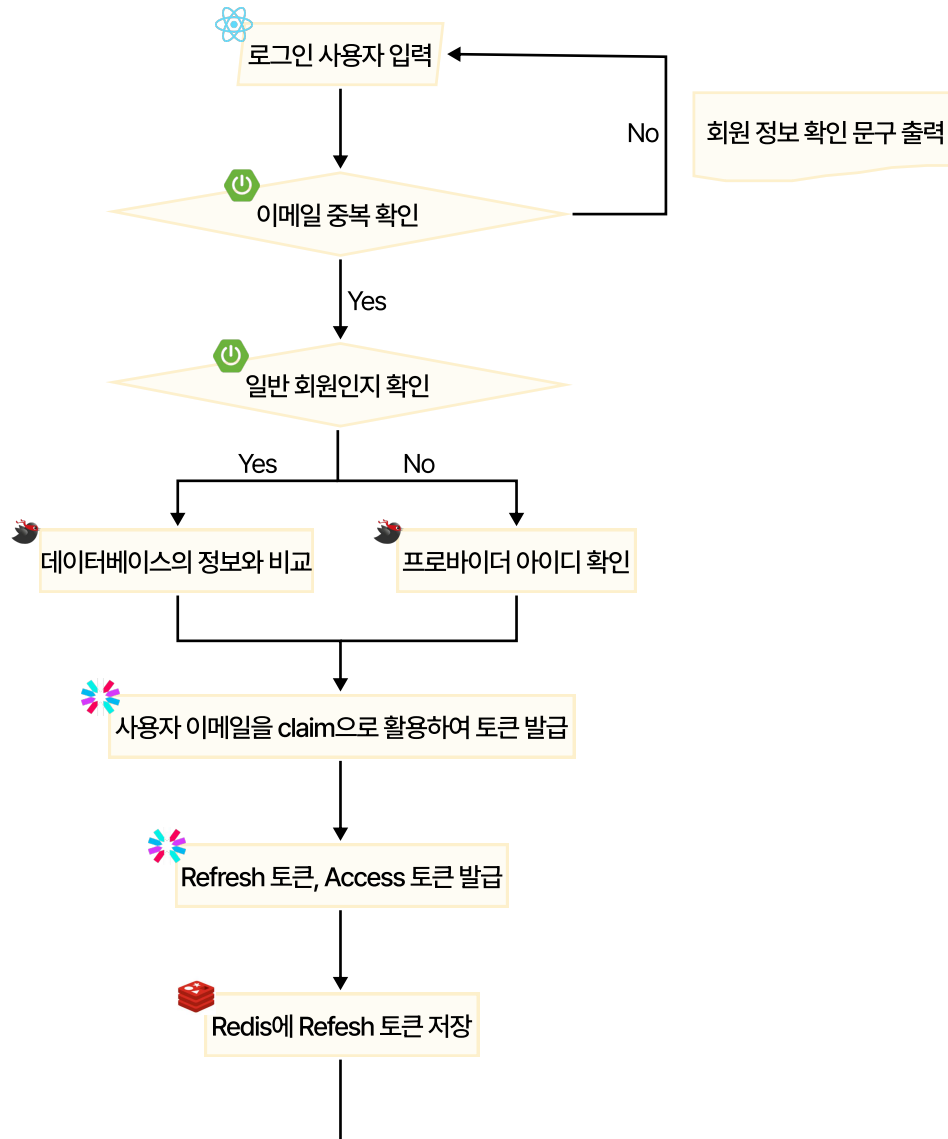


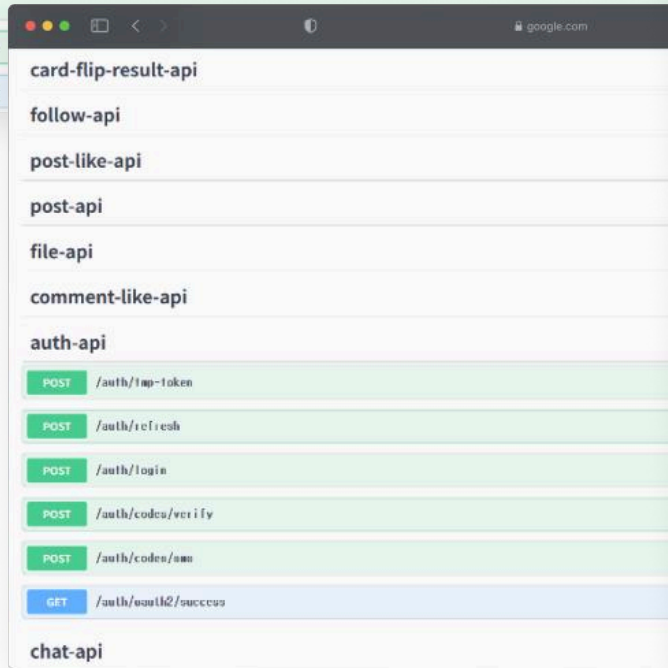
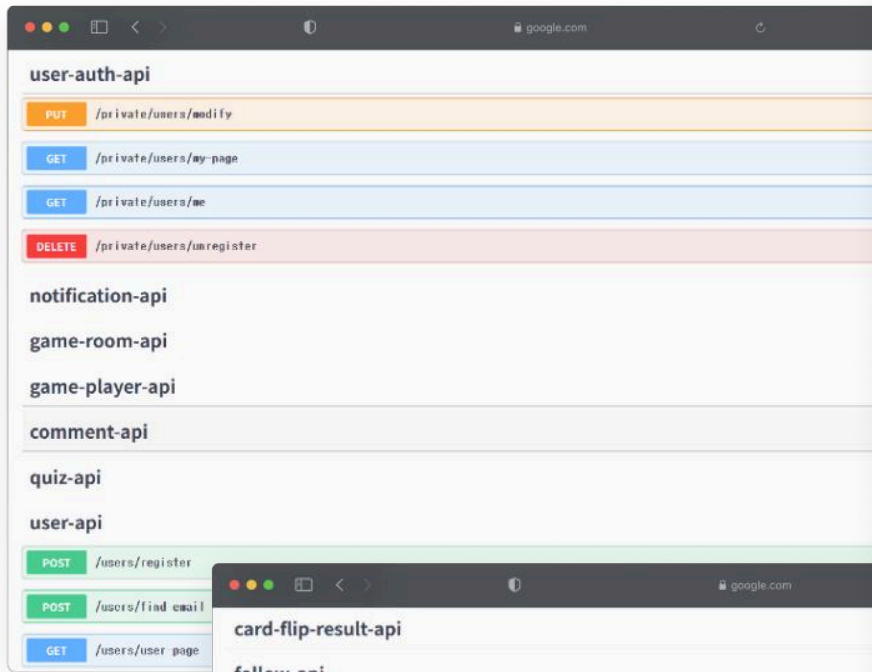
소셜 회원가입



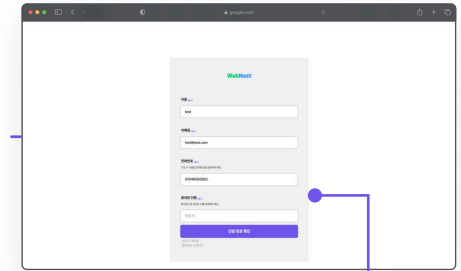
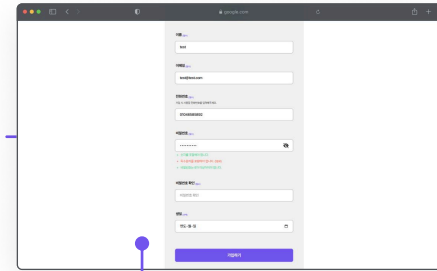
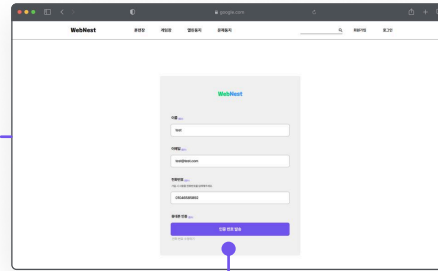
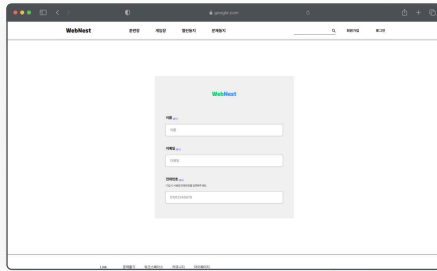


로그인





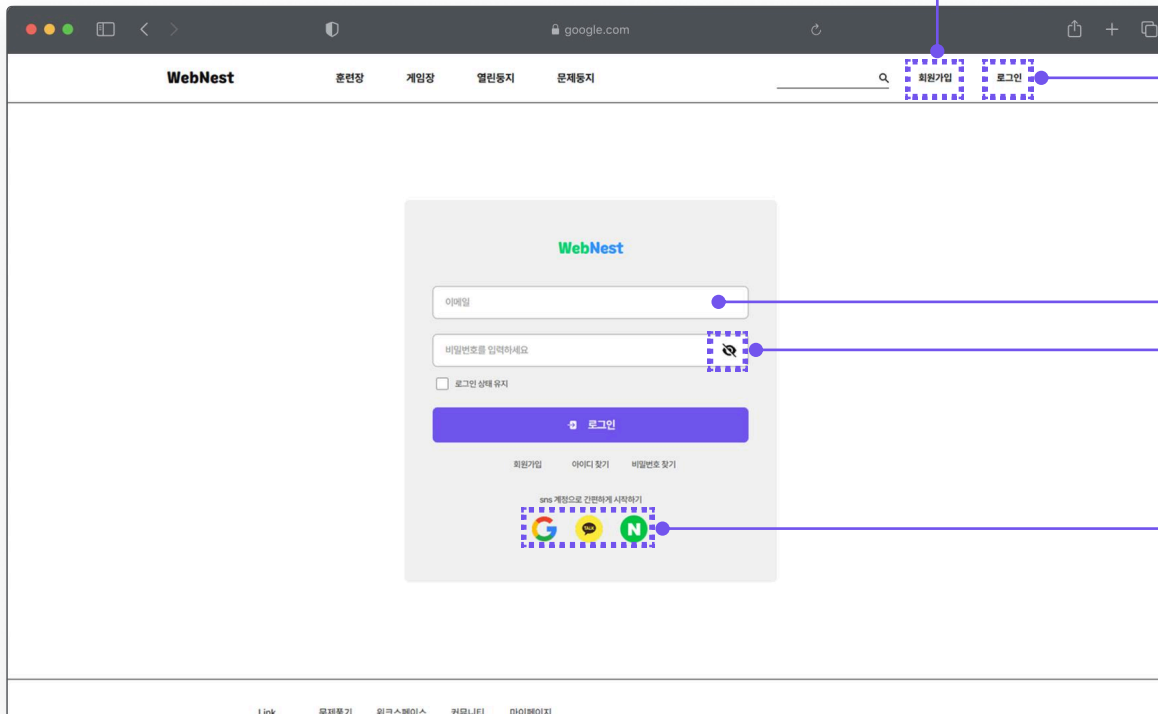
구분	요청경로	기능 설명	Method	인증	응답코드
회원가입	/users/register	목표: 신규 사용자 등록 및 회원가입 완료 주요 동작: 이메일 중복 검사 사용자 정보 저장 닉네임 없으면 이름을 닉네임으로 설정	POST	불필요 (Public)	201 CREATED
로그인	/auth/login	목표: 사용자 인증 후 JWT 토큰 발급 주요 동작: 이메일/비밀번호 검증 Access Token / Refresh Token 생성 Refresh Token: HttpOnly 쿠키 전송 Access Token: 응답 객체에 포함 Redis에 토큰 정보 저장	POST	불필요 (Public)	200 OK
임시 토큰 발급	/auth/tmp-token	목표: 비밀번호 찾기 과정에서 임시 Access Token 발급 주요 동작: UserVO 에서 전화번호 추출 해당 전화번호 사용자 조회 임시 Access Token 발급 후 반환 사용자 미존재 시 404 반환	POST	불필요 (Public)	200 OK 404 NOT_FOUND
이메일 찾기	/users/find-email	목표: 이름 + 전화번호로 이메일 조회 주요 동작: UserVO 에서 이름/전화번호 추출 일치 시 사용자 이메일 목록 조회 후 반환	POST	불필요 (Public)	200 OK
비밀번호 변경	/private/users/modify	목표: JWT 인증 사용자 정보 수정(비밀번호 포함 가능) 주요 동작: Authentication 에서 사용자 ID 추출 UserVO (Body) 기반 수정 처리 비밀번호 포함 시 암호화 후 저장 응답에서 비밀번호는 null 처리하여 제외	PUT	필요 (JWT)	200 OK
인게임 사용자 정보 업데이트	/player/{roomId}/{userId}	목표: 게임 중 플레이어 인게임 정보 업데이트 주요 동작: PathVariable roomId, userId 수신 RequestBody GameJoinDTO 수신 PathVariable ↔ Body 일치 검증 호스트 여부/팀 색상/턴/프로필 텍스트 업데이트	PUT	불필요 (Public)	200 OK
게임방 플레이어 조회	/player/{roomId}	목표: 특정 게임방 참여 플레이어 전체 조회 주요 동작: roomId 기반 플레이어 정보 반환 (닉네임/색상/턴/호스트 여부 등)	GET	불필요 (Public)	200 OK
구분	요청경로 (클라이언트 전송)	구독 경로 (서버 브로드캐스트)	기능 설명	Method	응답코드
단어 제출	/pub/game/last-word/submit	/sub/game/last-word/room/{roomId}	목표: 제출 단어 검증 + 단어 설명 생성(LLM) + 상태 브로드캐스트 주요 동작: 요청(Map)에서 userId, gameId, word 추출 유효성 검증(실제 단어/중복/체인 유지) LLM으로 단어 설명 생성 게임 상태와 함께 브로드캐스트	WebSocket @MessageMapping	200 OK
끝말잇기 준비	/pub/game/last-word/ready	/sub/game/last-word/room/{roomId}	목표: 게임 시작 전 플레이어 준비 상태 업데이트 및 방 전체 상태 브로드캐스트 주요 동작: GameJoinVO 수신 플레이어 준비 상태 업데이트 방 전체 플레이어 상태 조회 후 브로드캐스트	WebSocket @MessageMapping	200 OK
끝말잇기 시작	/pub/game/last-word/start	/sub/game/last-word/room/{roomId}	목표: 끝말잇기 게임 시작 및 첫 번째 플레이어 턴 부여 주요 동작: 전원 준비완료 처리 전원 턴 0 초기화 인덱스 0 플레이어 턴 1 설정 시작 상태 브로드캐스트	WebSocket @MessageMapping	200 OK
게임 상태 조회	/pub/game/last-word/state	/sub/game/last-word/room/{roomId}	목표: 현재 게임방 상태 조회 후 브로드캐스트 주요 동작: roomId 기반 상태 조회(플레이어 목록/턴 등) 상태 브로드캐스트	WebSocket @MessageMapping	200 OK



사용자 입력은 **정규식 표현법**과
리액트의 **옵셔널 문법**, **리액트 훅** 품을 활용

API를 활용하여 문자 전송
인증코드를 생성하여 사용자에게 전달 후
인증 완료 시 다음 단계로 진행

동적 스타일을 통해 비밀번호 인증 오류
발생 시 해당 오류에 대한 텍스트를
경고 색상으로 화면에 다시 표현



헤더를 통해 빠르게 회원가입 페이지로 이동

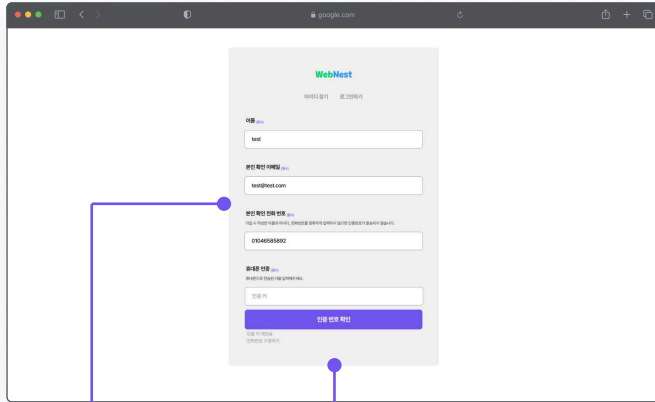
헤더를 통해 빠르게 로그인 페이지로 이동

리액트의 옵셔널과 리액트 훅 품 활용

해당 버튼을 통해 입력한 비밀번호가
무엇인지 텍스트타입으로 변환

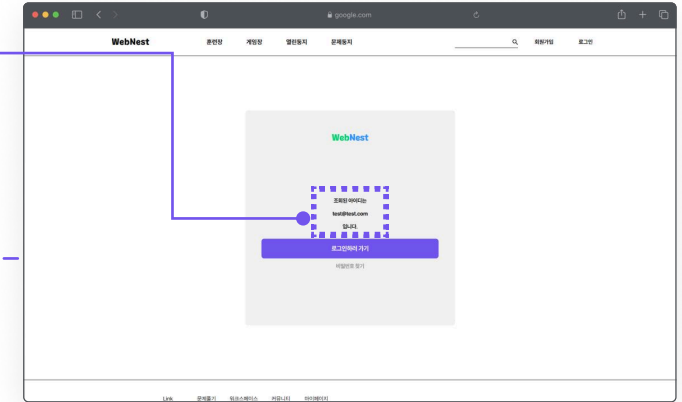
소셜로그인을 통한 사용자의 진입 문턱을 낮추고
토큰을 활용하여 일반 사용자와 일관성 있게 관리

본인 인증 절차와 임시 토큰을 활용하여 개인 정보를 보호
ID 찾기와 PW 찾기의 다른 성격을 활용하여 서로 다른 결과창 출력



The form is titled 'WebNest' and '아이디 찾기 | 비밀번호 찾기'. It has two main sections: '아이디 찾기' and '비밀번호 찾기'. The '아이디 찾기' section includes a text input for '이름', a text input for '본인 확인 이메일', a text input for '본인 확인 전화 번호', and a text input for '인증 코드'. The '비밀번호 찾기' section includes a text input for '이름' and a text input for '인증 코드'. A blue button labeled '인증 코드 확인' is at the bottom.

ID 찾기의 경우 사용자의 아이디가
여러 개 일 수 있기 때문에
리스트의 형태로 모든 아이디들만 화면에 표현

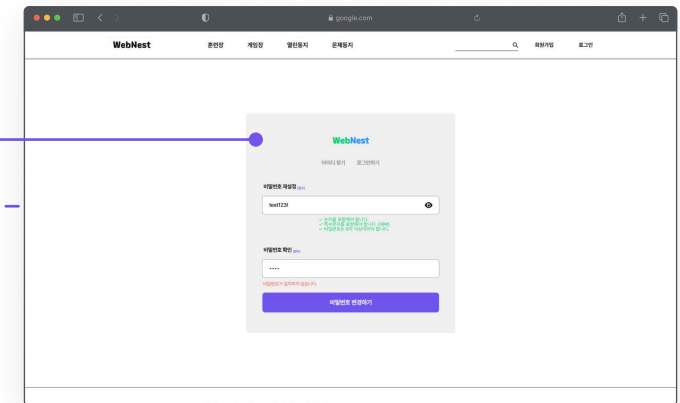


The page shows the 'WebNest' header and navigation links. A search bar is at the top right. Below it, a form titled 'WebNest' and '아이디 찾기 | 비밀번호 찾기' is displayed. The '아이디 찾기' section shows a list of search results for the email 'test@web.com'. A blue button labeled '로그인하여 찾기' is at the bottom.

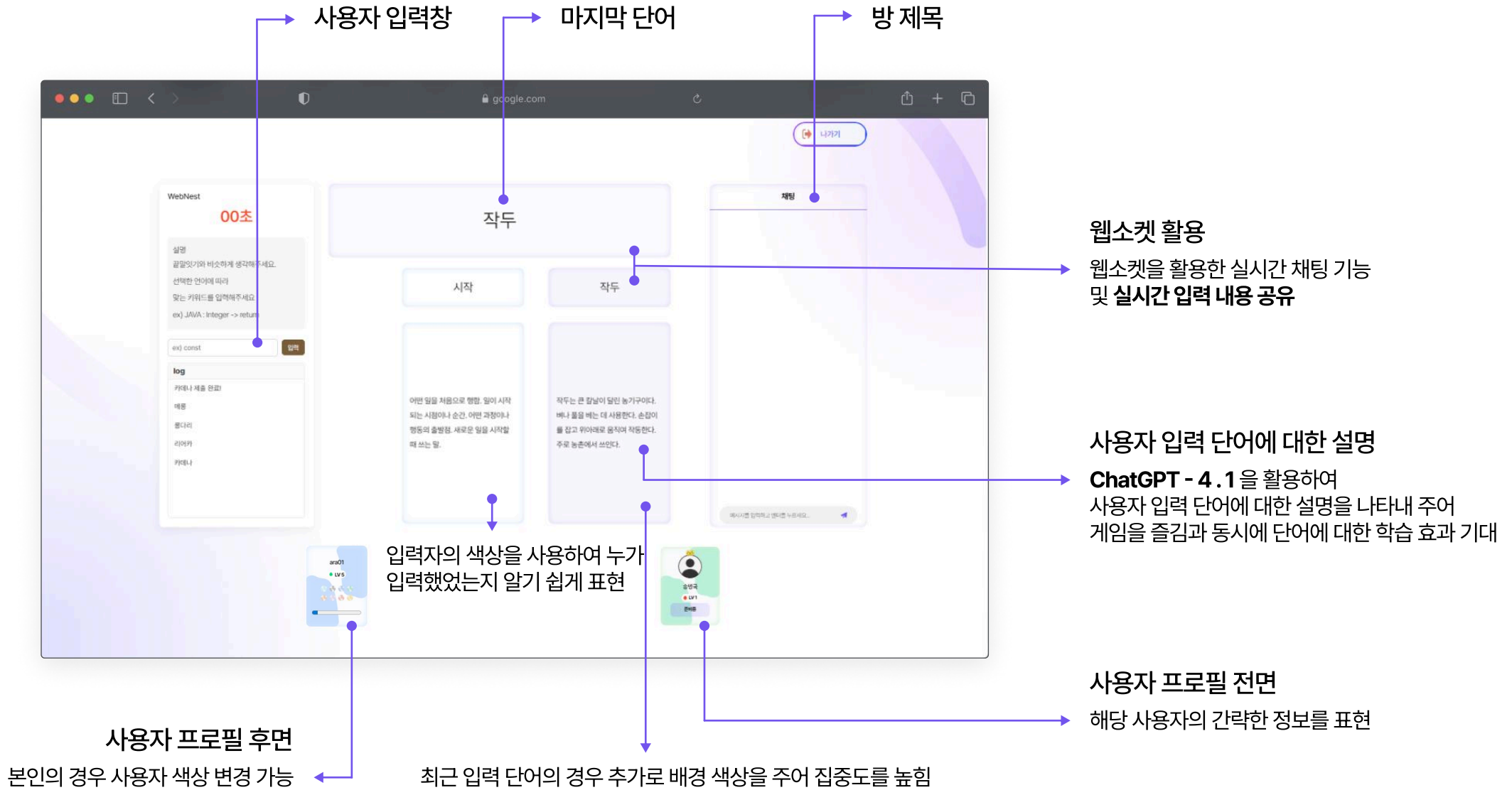
정규식 표현법과 리엑트의 옵셔널 문법,
리엑트 후 폼을 활용하여
사용자의 입력값과 데이터베이스를 비교 후,
사용자가 원하는 서비스를 사용할 수 있도록 제공

API를 활용하여 문자 전송
인증코드를 생성하여 사용자에게 전달 후
인증 완료 시 다음 단계로 진행

PW찾기의 경우 회원정보 인증 완료 시 임시 토큰을 발급 받아
해당 임시 토큰을 활용하여 사용자의 PW 변경 서비스를 제공



The page shows the 'WebNest' header and navigation links. A search bar is at the top right. Below it, a form titled 'WebNest' and '아이디 찾기 | 비밀번호 찾기' is displayed. The '비밀번호 찾기' section shows a form for password reset. It includes a text input for '이름', a text input for '인증 코드', and a text input for '비밀번호'. A blue button labeled '비밀번호 변경하기' is at the bottom.





상황

LLM 사용 시 데이터를 가져오지 못했는데에도 다음 코드가 실행되는 상황

해결방법

Map을 통한 Cash 변수를 생성하여 값을 저장했습니다.
최대 3번까지 요청하며, 필요한 값인 단어와 설명 부분을
Cash에 저장합니다.
값을 정확하게 가지고 왔다면 설명을 바로 리턴해주고,
값을 가져오지 못한 경우 확인을 위해 문자열 메시지를
바로 응답해 주었습니다.

해당 경험을 통해 알게된 점

API 사용 시 정해진 요청 경로에 정해진 요청 값을 정해진 타입으로
요청해야 하고, 응답을 받는 경우에도 정해진 이름과 타입으로 응답을
받아야 한다는 것을 알았습니다.
이전 OAuth2.0의 경우와 마찬가지로 값이 응답되기 전에 다음 로직을
실행시켜버리는 비동기 문제가 발생 할 수 있기에 그 부분도 유의하며
로직을 작성해야했습니다.
따라서 API를 사용할 경우, 요청을 위한 객체와 응답을 위한 객체
두가지가 필요하다는 것을 알게 되었습니다.

```

for (int attempt = 1; attempt <= 3; attempt++) {
    try {
        WordExplanationResponseDTO response = openAiWebClient.post()
            .uri(uri: "/chat/completions")
            .bodyValue(request)
            .retrieve()
            .bodyToMono(WordExplanationResponseDTO.class)
            .block(); // 동기 호출

        if (response == null ||
            response.getChoices() == null ||
            response.getChoices().isEmpty() ||
            response.getChoices().get(0).getMessage() == null ||
            response.getChoices().get(0).getMessage().getContent() == null) {
            continue; // 다음 재시도
        }

        String explanation = response.getChoices()
            .get(0)
            .getMessage()
            .getContent()
            .trim();

        cache.put(word, explanation); // 캐시에 저장
        return explanation;
    } catch (WebClientResponseException e) {
        int status = e.getRawStatusCode();
        if (status == 429 || (status >= 500 && status < 600)) {
            sleepBackoff(attempt);
        } else {
            break;
        }
    } catch (Exception e) {
        sleepBackoff(attempt);
    }
}

return "단어 설명을 가져오지 못했습니다.";
}

private void sleepBackoff(int attempt) {
    try {
        Thread.sleep(200L * attempt); // 0.2초, 0.4초, 0.6초
    } catch (InterruptedException ie) {
        Thread.currentThread().interrupt();
    }
}

```



상황

소셜 회원 가입 시 기본 입력값이 정확하게 VO로 전달되지 않는 상황

해결방법

OAuth2.0를 통해 들어오는 값을 데이터베이스에 저장하기 위해 만든 VO에 저장할 때, 닉네임 또는 Provider 이름을 사용하여 중복되지 않는 임의의 값을 저장해두었습니다.

그 후 데이터베이스에 사용자 등록을 마친 다음 바로 로그인이 적용되도록 해주었습니다.

해당 경험을 통해 알게된 점

OAuth2.0의 경우 모든 사이트의 값이 동일한 값으로 전달되는 것이 아닌, 각 사이트에 정해진 이름으로 값을 전달해 주었습니다.

또한, 값을 전달받는 즉시 이름을 바꿔 저장하는 경우, 비동기 문제가 발생하여 값이 제대로 저장되지 않을 수 있다는 것을 알게 되었습니다.

이를 통해 추후 값을 요청하고 응답하는 경우, 해당 로직이 동기적인지 비동기적인지 한차례 더 생각하게 되는 계기가 되었습니다.

```
UserVO newUser = new UserVO(userInsertSocialVO);
if (newUser.getUserName() == null || newUser.getUserName().isBlank()) {
    String fallback = (newUser.getUserNickname() != null && !newUser.getUserNickname().isBlank())
        ? newUser.getUserNickname()
        : (newUser.getUserProvider() != null ? newUser.getUserProvider() : "SOCIAL") + "_USER_" +
        java.util.UUID.randomUUID().toString().substring(0,6);
    newUser.setUserName(fallback);
    if (newUser.getUserNickname() == null || newUser.getUserNickname().isBlank()) {
        newUser.setUserNickname(fallback);
    }
}
userDAO.save(newUser);

String userEmail = userInsertSocialVO.getUserEmail();
Long userId = userDAO.findIdByEmail(userEmail);
claim.put("userEmail", userEmail);
String refreshToken = jwtTokenUtil.generateRefreshToken(claim);
String accessToken = jwtTokenUtil.generateAccessToken(claim);

userSocialVO.setUserId(userId);
userService.registerUserSocial(userSocialVO);

tokens.put("accessToken", accessToken);
tokens.put("refreshToken", refreshToken);

return tokens;
```



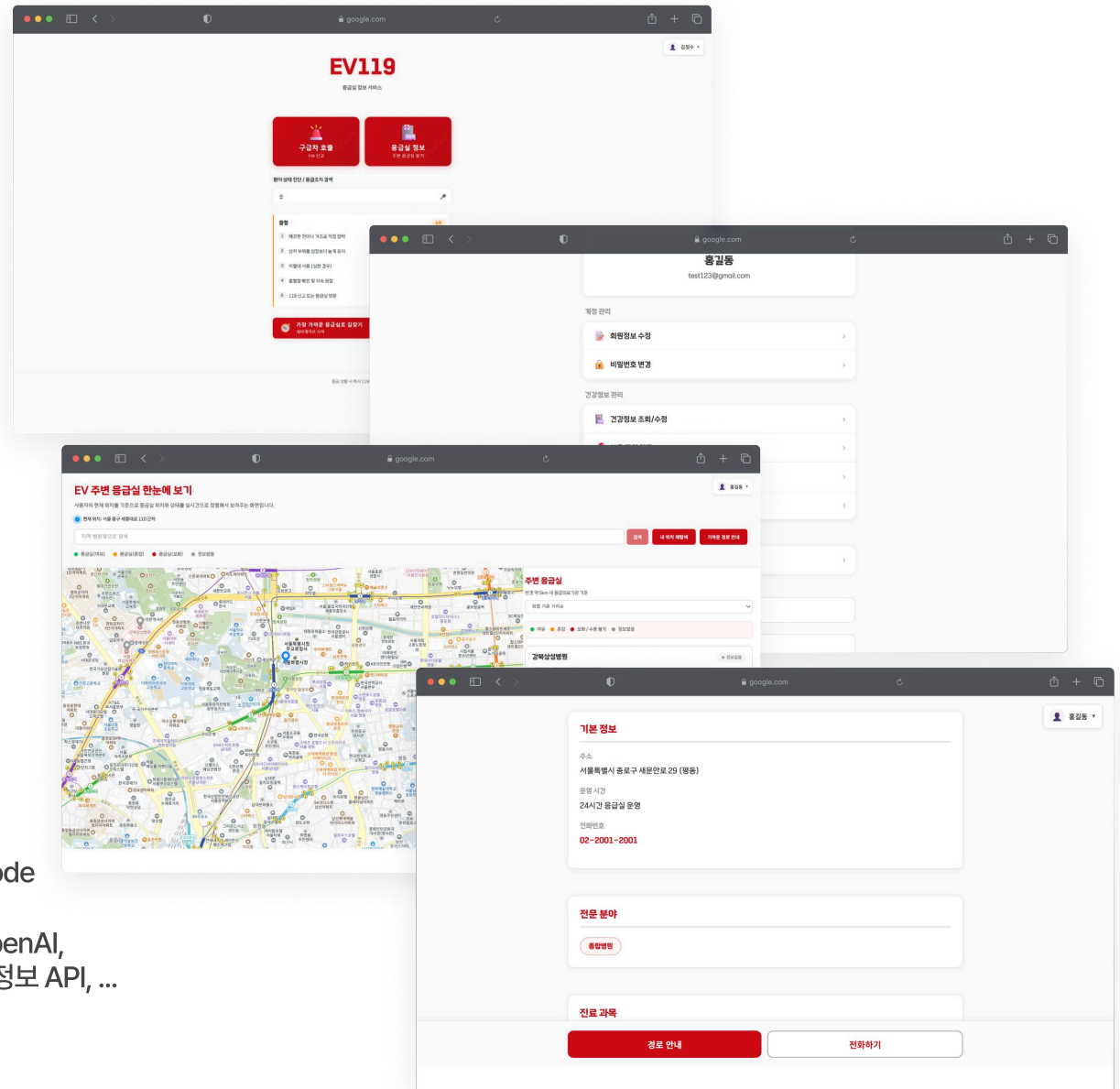

EV119

공공데이터를 활용하여 빠르게 응급실에 도착하고
빅데이터를 활용하여 간단한 응급처치에 대한 조언을 구할 수 있는 서비스

개발 기간	2025. 11 ~ 2025. 12 (약 1개월)
플랫폼	Web
개발 인원	4명 (팀원)
담당 역할	마이페이지 - 회원정보 수정, 비밀번호 변경 마이페이지 - 건강정보 조회 및 관리 마이페이지 - 과거 병원 방문이력 마이페이지 - 회원 탈퇴

Development Environment

Language	Java (JDK 17), HTML, CSS, JavaScript
Server, Cloud	Apache Tomcat 9.0
Framework	Spring Boot 3.2.x, React 18.x
DB	Oracle 21C, Redis 8.x
IDE	IntelliJ IDEA 2025.2.3, Visual Studio Code
API, Library	JPA, OAuth2, Spring Security, JWT, OpenAI, 카카오 맵 API, 응급실 정보 API, 외상센터 정보 API, ...
DevOps, Tools	Git, GitHub, Docker

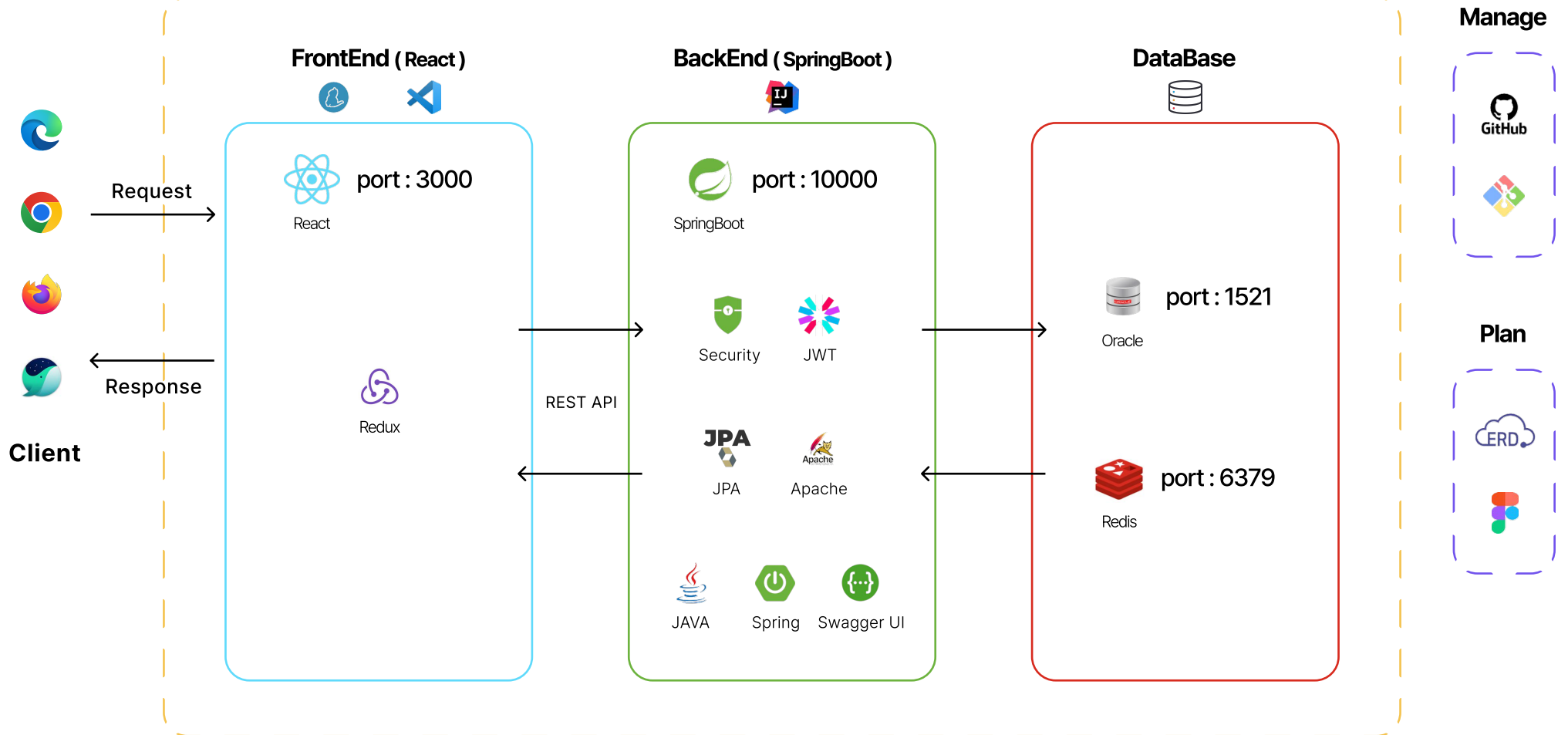




EV119

| 시스템 구성도

외부 API





Font: The Jamsil

The Jamsil Thin

가A

The Jamsil Regular

가A

The Jamsil Bold

가A

The Jamsil Light

가A

The Jamsil Medium

가A

The Jamsil Medium

가A

COLOR

Primary

Name primary
Hex #B80F16
rgb 184, 15, 22



Secondary

Name secondary
Hex #CD0B16
rgb 205, 11, 22



Deactivated

Name deactivate
Hex #CE7E7A
rgb 206, 126, 122



UserLocation

Name primary
Hex #2196F3
rgb 33, 150, 243



Available

Name available
Hex #6BC462
rgb 107, 196, 98



Primary

Name primary
Hex #FF9800
rgb 255, 152, 0



Usage: 폰트 크기 및 자족

Heading1

48px, bold

Heading3

28px, bold

Title

20px, medium

Paragraph

14px, regular

Heading2

36px, bold

Heading4

24px, medium

Subtitle

16px, medium

Paragraph Strong

14px, medium

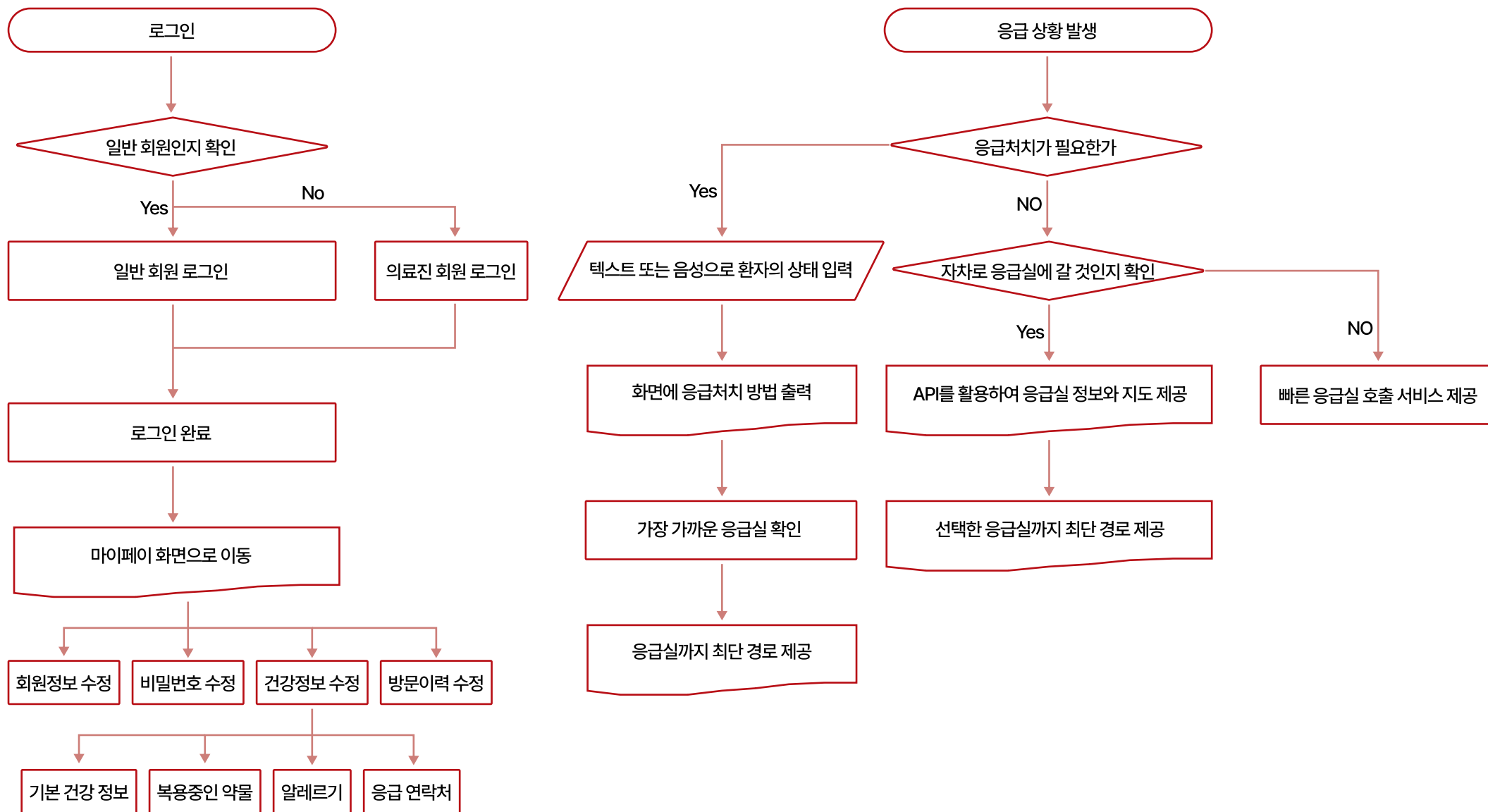
Button Type

	Button Default	Button Primary
Normal		
Hover		
Disabled		



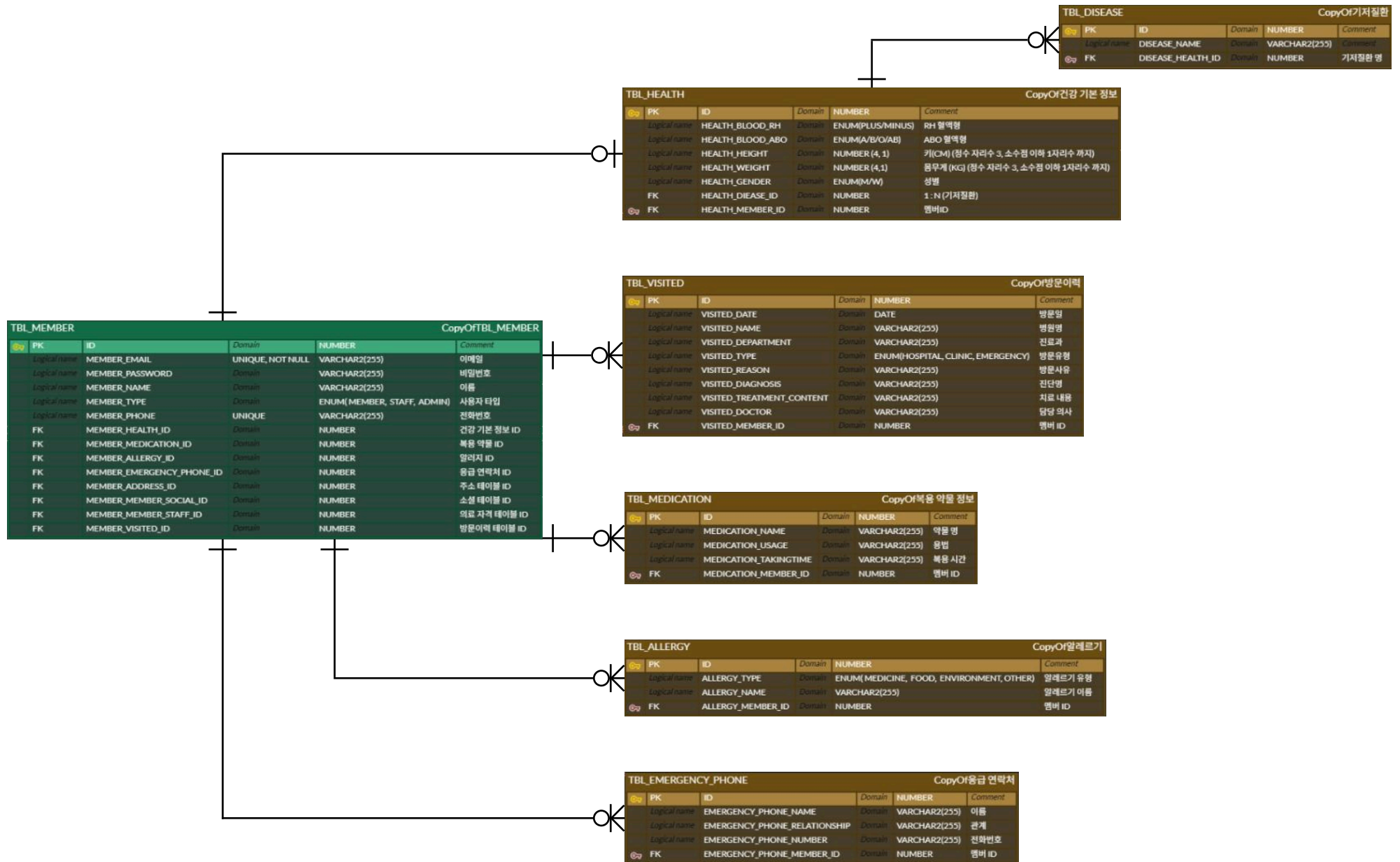
EV119

| 서비스 흐름도



**EV119**

서비스 설계 (ERDCloud - 마이페이지)





EV119

| 서비스 설계 (기능 명세서)

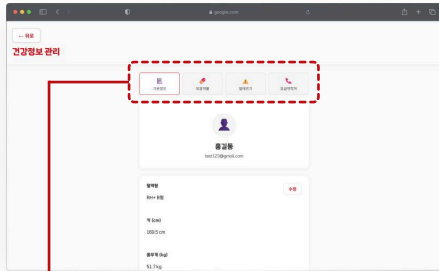
my-page-api	
PUT	/my-page/edit
POST	/my-page/change-password
GET	/my-page/me
DELETE	/my-page/unregister
sms-api	
POST	/sms/verify
POST	/sms/send
visited-api	
POST	/my-page/visited/add
GET	/my-page/visited
DELETE	/my-page/visited/delete
medication-api	
POST	/my-page/medication/modify
GET	/my-page/medication
health-api	
POST	/my-page/health/modify
POST	/my-page/health/add-disease
GET	/my-page/health
DELETE	/my-page/health/remove-disease
emergency-phone-api	
POST	/my-page/emergency-phone/modify
GET	/my-page/emergency-phone
allergy-api	
POST	/my-page/allergy/modify
GET	/my-page/allergy

구분	요청 경로	기능 설명	Method	인증	응답 코드
프로필 조회	/my-page/me	목표: JWT 인증 사용자의 프로필 정보 조회 주요 동작: authentication 에서 사용자 ID 추출, 사용자 프로필 정보 조회 후 반환	GET	필요 (JWT)	200 OK
프로필 수정	/my-page/edit	목표: JWT 인증 사용자의 프로필 정보 수정 주요 동작: authentication 에서 사용자 ID 추출, MemberDTO (Body) 기반 수정 쿼리, 수정된 프로필 정보 반환	PUT	필요 (JWT)	200 OK
비밀번호 변경	/my-page/change-password	목표: JWT 인증 사용자의 비밀번호 변경 주요 동작: authentication 에서 사용자 ID 추출, ChangePasswordDTO (Body) 기반 비밀번호 변경, 비밀번호 암호화 후 저장	POST	필요 (JWT)	200 OK
회원 탈퇴	/my-page/unregister	목표: JWT 인증 사용자의 회원 탈퇴 처리 주요 동작: authentication 에서 사용자 ID 추출, 사용자 정보 삭제 처리	DELETE	필요 (JWT)	204 NO CONTENT
알러지 정보 조회	/my-page/allergy	목표: JWT 인증 사용자의 알러지 정보 조회 주요 동작: authentication 에서 사용자 ID 추출, 사용자의 알러지 목록 조회 후 반환	GET	필요 (JWT)	200 OK
알러지 정보 수정	/my-page/allergy/modify	목표: JWT 인증 사용자의 알러지 정보 수정 주요 동작: authentication 에서 사용자 ID 추출, List<AllergyDTO> (Body) 기반 알러지 정보 수정, 수정된 알러지 목록 반환	POST	필요 (JWT)	200 OK
응급 연락처 조회	/my-page/emergency-phone	목표: JWT 인증 사용자의 응급 연락처 정보 조회 주요 동작: authentication 에서 사용자 ID 추출, 사용자의 응급 연락처 목록 조회 후 반환	GET	필요 (JWT)	200 OK
응급 연락처 수정	/my-page/emergency-phone/modify	목표: JWT 인증 사용자의 응급 연락처 정보 수정 주요 동작: authentication 에서 사용자 ID 추출, List<EmergencyPhoneDTO> (Body) 기반 응급 연락처 정보 수정, 수정된 응급 연락처 목록 반환	POST	필요 (JWT)	200 OK
건강 정보 조회	/my-page/health	목표: JWT 인증 사용자의 건강 정보 조회 주요 동작: authentication 에서 사용자 ID 추출, 사용자의 건강 정보 조회 후 반환	GET	필요 (JWT)	200 OK
건강 정보 수정	/my-page/health/modify	목표: JWT 인증 사용자의 건강 정보 수정 주요 동작: authentication 에서 사용자 ID 추출, HealthDTO (Body) 기반 건강 정보 수정, 수정된 건강 정보 반환	POST	필요 (JWT)	200 OK
기저질환 추가	/my-page/health/add-disease	목표: JWT 인증 사용자의 기저질환 추가 주요 동작: authentication 에서 사용자 ID 추출, diseaseName 파라미터 기반 기저질환 추가, 추가된 건강 정보 반환	POST	필요 (JWT)	200 OK
기저질환 삭제	/my-page/health/remove-disease	목표: JWT 인증 사용자의 기저질환 삭제 주요 동작: authentication 에서 사용자 ID 추출, DiseaseDTO (Body) 기반 기저질환 삭제, 삭제된 건강 정보 반환	DELETE	필요 (JWT)	200 OK
복용 약물 정보 조회	/my-page/medication	목표: JWT 인증 사용자의 복용 약물 정보 조회 주요 동작: authentication 에서 사용자 ID 추출, 사용자의 복용 약물 목록 조회 후 반환	GET	필요 (JWT)	200 OK
복용 약물 정보 수정	/my-page/medication/modify	목표: JWT 인증 사용자의 복용 약물 정보 수정 주요 동작: authentication 에서 사용자 ID 추출, List<MedicationDTO> (Body) 기반 복용 약물 정보 수정, 수정된 복용 약물 목록 반환	POST	필요 (JWT)	200 OK
방문 이력 조회	/my-page/visited	목표: JWT 인증 사용자의 방문 이력 정보 조회 주요 동작: authentication 에서 사용자 ID 추출, 사용자의 방문 이력 목록 조회 후 반환	GET	필요 (JWT)	200 OK
방문 이력 추가	/my-page/visited/add	목표: JWT 인증 사용자의 방문 이력 추가 주요 동작: authentication 에서 사용자 ID 추출, VisitedDTO (Body) 기반 방문 이력 추가, 추가된 방문 이력 목록 반환	POST	필요 (JWT)	200 OK
방문 이력 삭제	/my-page/visited/delete	목표: JWT 인증 사용자의 방문 이력 삭제 주요 동작: authentication 에서 사용자 ID 추출, VisitedDTO (Body) 기반 방문 이력 삭제, 삭제된 방문 이력 목록 반환	DELETE	필요 (JWT)	200 OK

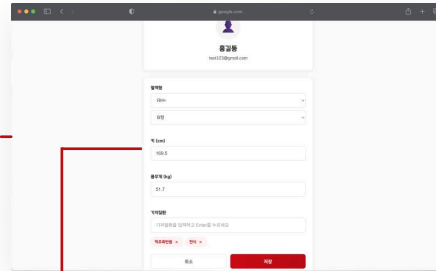


EV119

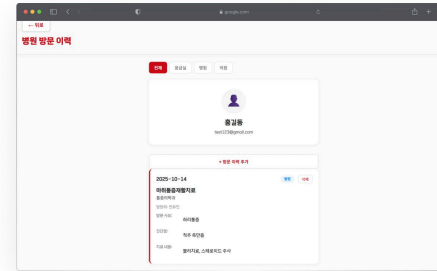
| 화면 | 건강정보 관리



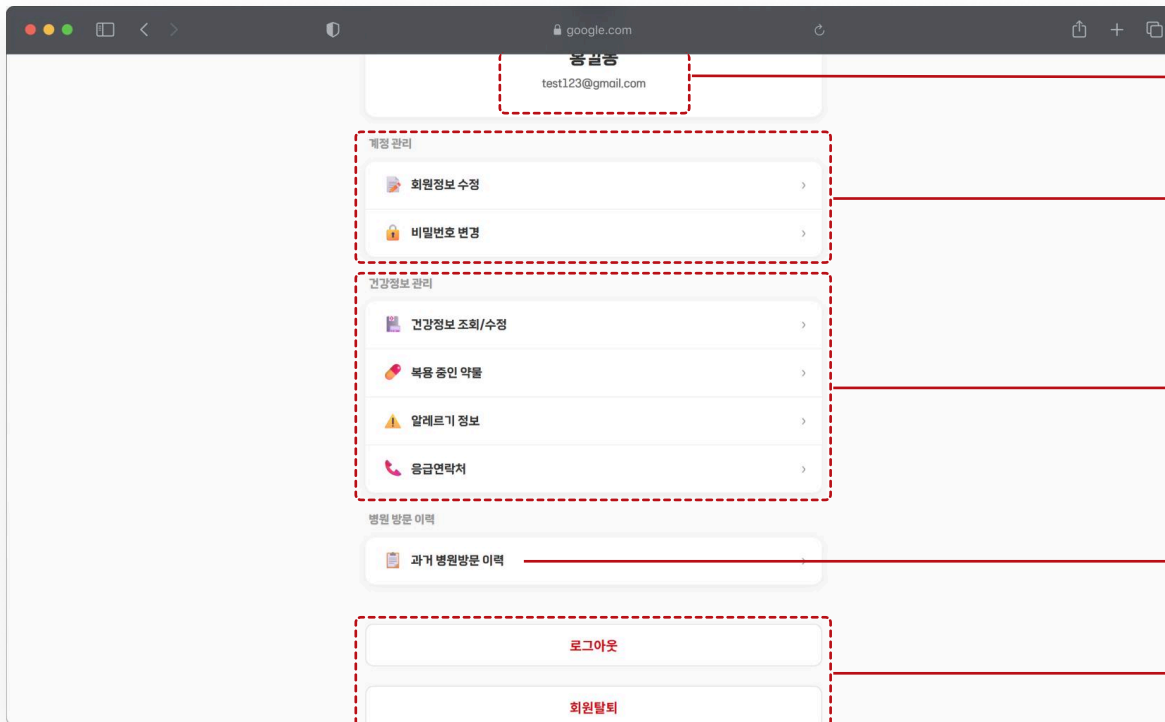
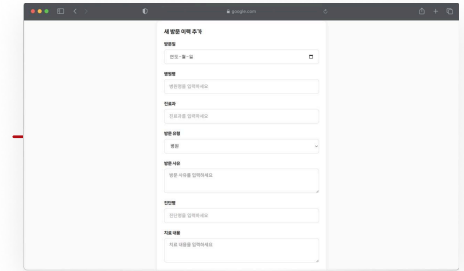
각 정보를 카테고리 별로 분류
선택 시 내용을 포함하고 있는
컴포넌트만 리랜더링



수정 선택 시 기존의 내용이 들어가 있는 상태로
사용자의 수정 후 완료 버튼 클릭 시
수정된 정보를 저장



방문 이력의 경우
각각의 방문 이력을 따로 수정, 삭제 할 수 있도록 제공
방문일은 미래의 시간을 선택 할 수 없도록 제한



마이페이지의 상단 부분을 통해 현재 로그인 한 계정의
사용자 이름과 접속한 이메일을 확인 가능

계정 관리 부분을 통해 해당 계정의 기본적인 정보와
비밀번호 변경 서비스 제공

건강정보 관리 부분을 통해 해당 사용자의 건강 관련 정보를
읽기, 수정, 삭제 가 가능하도록 제공
각 버튼을 통해 해당 컴포넌트를 바로 제공

과거 병원 방문이력은 각 병원의 종류를 구분하는 카테고리를 두어
건강정보와 별도로 확인 할 수 있도록 분리

마이페이지 내부에서 회원탈퇴와 로그아웃을 할 수 있도록
기능 제공



회원 가입 시 입력받는 가장 기본적인 사용자의 정보를 확인할 수 있는 페이지

전화번호 입력의 경우 정규식 표현을 활용하여 숫자만 입력 할 수 있도록 확인
각 전화번호 사이에 하이픈 (-)을 자동으로 표시해두어 사용자가 확인 하기 쉽도록 표현
데이터 입력 시엔 각 하이픈이 없는 숫자들로 이루어진 문자열만 전송

이메일 입력의 경우 정규식 표현을 활용하여 이메일 형식의 문자열을 받도록 확인

비밀번호 변경 시

1. 새 비밀번호가 사용자의 원하는 값으로 정확히 입력된 값인지 확인
2. 현재 비밀번호와 로그인된 액세스 토큰으로 사용자를 확인
3. 새 비밀번호를 확인된 사용자 DB에 업데이트